FPT UNIVERSITY

MÔN HỌC: EMBEDDED LINUX SYSTEMS (LOS201)

BÁO CÁO THỰC HÀNH

LAB 01

CẤU HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG LINUX NHÚNG (ARMv7)

ARMv7-A Cortex-A9 | QEMU vexpress-a9 | Linux Kernel 5.15 LTS

Họ và tên - MSSV	Phạm Lê Hải Đăng – SE201613 Phan Đặng Ái Trực – SE201700 Nguyễn Trần Quang Huỳnh Nhân Thiện Trí Tín
Nhóm/Lớp	Nhóm 1 / IC2008
Giảng viên	Phạm Thế Vinh
Ngày thực hiện	14/09/2026

Fall 2026

1. THÔNG TIN BÀI LAB VÀ MỤC TIÊU

1.1. Thông tin bài lab

Môn học	Embedded Linux Systems (LOS201)
Mã bài lab	LAB-01
Tên bài lab	Cấu hình Kernel & Khởi động Hệ thống Linux Nhúng (ARMv7)
Kiến trúc mục tiêu	ARMv7-A - Cortex-A9
Nền tảng giả lập	QEMU vexpress-a9
Linux Kernel	5.15 LTS
U-Boot	2022.04
BusyBox	1.35.0

1.2. Mục tiêu

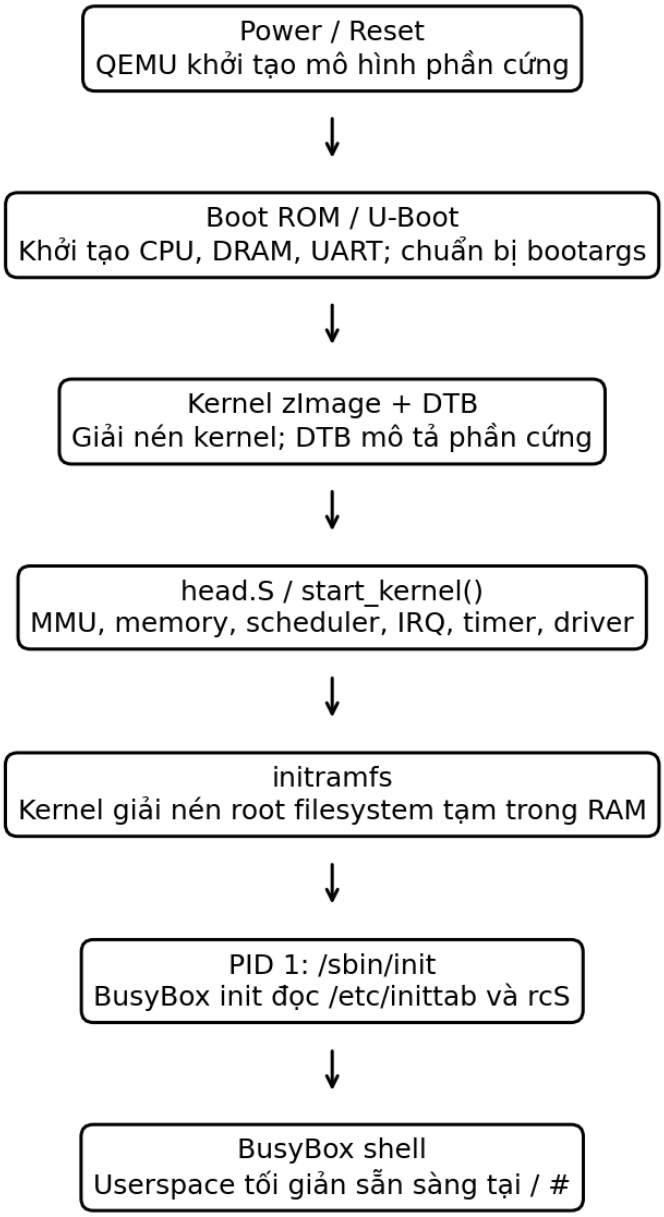
- Hiểu chuỗi khởi động của một hệ thống Embedded Linux từ bootloader đến userspace.
- Cross-compile Linux kernel cho kiến trúc ARMv7-A và tạo zImage, Device Tree Blob (DTB).
- Cấu hình và sử dụng U-Boot, đặc biệt là bootargs và bootcmd.
- Xây dựng Root Filesystem tối giản bằng BusyBox và đóng gói thành initramfs.
- Boot hệ thống Linux hoàn chỉnh trên QEMU vexpress-a9 và kiểm tra CPU, memory, filesystem, process.
- Phân tích kernel log, khoanh vùng nguyên nhân khi build/boot lỗi và trình bày phương pháp debug có hệ thống.

1.3. Môi trường thực hiện

Thành phần	Yêu cầu/Thiết lập	Kết quả thực tế
Host OS	Ubuntu Linux 64-bit	Ubuntu 26.04.1 LTS
Cross toolchain	arm-linux-gnueabihf-gcc	arm-linux-gnueabihf-gcc 15.2.0
QEMU	qemu-system-arm	QEMU 10.2.1
Target CPU	Cortex-A9 / ARMv7-A	QEMU vexpress-a9
RAM mô phỏng	512 MiB	512 MiB
Kernel	Linux 5.15	Đã build zImage thành công
DTB	vexpress-v2p-ca9.dtb	Đã build thành công, ~14 KB

2. KIẾN TRÚC HỆ THỐNG VÀ BOOT SEQUENCE

Bài lab mô phỏng chuỗi khởi động: Power/Reset → Boot ROM/U-Boot → kernel zImage (nhận DTB và kernel command-line) → giải nén initramfs → PID 1 (/sbin/init) → BusyBox shell.



Hình 1. Boot Sequence tổng quát của hệ thống ARMv7 Cortex-A9 trong Lab 01

2.1. Vai trò của từng thành phần

Thành phần	Vai trò	Thông tin truyền sang bước sau
QEMU / Machine model	Mô phỏng CPU Cortex-A9, RAM và các peripheral của Versatile Express.	Môi trường phần cứng cho bootloader/kernel.
U-Boot	Khởi tạo phần cứng sớm và chuẩn bị image/boot parameters.	Địa chỉ kernel, DTB, initramfs và kernel command-line.

Thành phần	Vai trò	Thông tin truyền sang bước sau
zImage	Kernel ARM32 đã nén; tự giải nén và chuyển vào kernel entry.	Điều khiển CPU, MMU, memory, scheduler, IRQ và driver.
DTB	Mô tả topology phần cứng: CPU, RAM, UART, interrupt controller...	Kernel parse DTB để khởi tạo driver/platform.
initramfs	Root filesystem tạm nằm trong RAM.	Cung cấp /sbin/init, /bin/sh, config và device nodes.
BusyBox init	PID 1 trong userspace, đọc /etc/inittab và chạy rcS.	Khởi tạo proc/sysfs/tmpfs và mở shell.

3. PHẦN A - CẤU HÌNH VÀ BUILD LINUX KERNEL

3.1. Thiết lập cross-compilation

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-

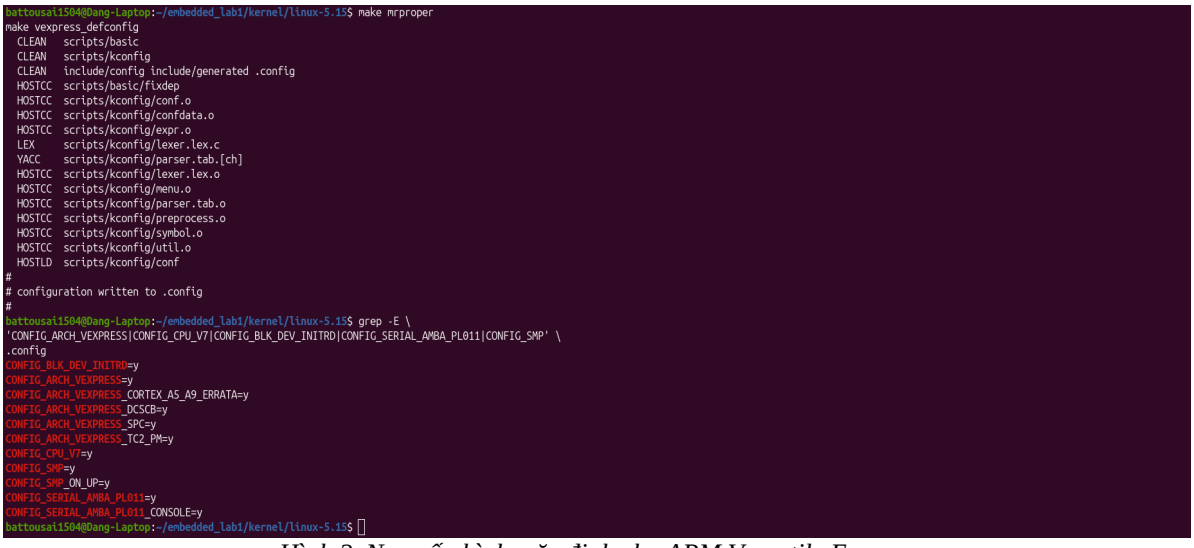
# Không ép hard-float flags vào toàn bộ kernel
unset KCFLAGS KBUILD_CFLAGS CFLAGS CXXFLAGS CPPFLAGS ASFLAGS
```

ARCH=arm yêu cầu hệ thống build tạo kernel ARM32. CROSS_COMPILE chọn prefix của ARM GNU toolchain. Kernel build system sẽ tự chọn các option CPU/ISA dựa trên .config và arch/arm/Makefile.

3.2. Nạp cấu hình VExpress

```
make vexpress_defconfig
grep -E "CONFIG_ARCH_VEXPRESS|CONFIG_CPU_V7|CONFIG_CPU_32v7|CONFIG_SMP" .config
```

Target của bài lab là QEMU vexpress-a9, do đó sử dụng vexpress_defconfig. Cấu hình cần có hỗ trợ ARMv7 và platform VExpress; SMP được bật nếu muốn mô phỏng nhiều CPU.



Hình 2. Nạp cấu hình mặc định cho ARM Versatile Express

3.3. Build zImage và Device Tree

```
make -j"$(nproc)" zImage
make -j"$(nproc)" dtbs

ls -lh arch/arm/boot/zImage
ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
```

Kết quả thực tế hiện tại: **zImage đã build thành công** và **vexpress-v2p-ca9.dtb đã build thành công (~14 KB)**. DTB đúng với board model vexpress-a9/Cortex-A9 mà QEMU sẽ sử dụng.

```
/usr/bin/arm-linux-gnueabihf-ld.bfd: warning: vmlinux has a LOAD segment with RWX permissions
  SORTTAB vmlinux
  SYSMAP  System.map
  OBJCOPY arch/arm/boot/Image
  Kernel: arch/arm/boot/Image is ready
  LDS     arch/arm/boot/compressed/vmlinux.lds
  AS      arch/arm/boot/compressed/head.o
  GZIP    arch/arm/boot/compressed/piggy_data
  CC      arch/arm/boot/compressed/misc.o
  CC      arch/arm/boot/compressed/decompress.o
  CC      arch/arm/boot/compressed/string.o
  SHIPPED arch/arm/boot/compressed/hyp-stub.S
  CC      arch/arm/boot/compressed/fdt_rw.o
  CC      arch/arm/boot/compressed/fdt_ro.o
  CC      arch/arm/boot/compressed/fdt_wip.o
  CC      arch/arm/boot/compressed/fdt.o
  CC      arch/arm/boot/compressed/fdt_check_mem_start.o
  SHIPPED arch/arm/boot/compressed/lib1funcs.S
  SHIPPED arch/arm/boot/compressed/ashldi3.S
  SHIPPED arch/arm/boot/compressed/bswapsdi2.S
  AS      arch/arm/boot/compressed/piggy.o
  AS      arch/arm/boot/compressed/hyp-stub.o
  AS      arch/arm/boot/compressed/lib1funcs.o
  AS      arch/arm/boot/compressed/ashldi3.o
  AS      arch/arm/boot/compressed/bswapsdi2.o
  LD      arch/arm/boot/compressed/vmlinux
/usr/bin/arm-linux-gnueabihf-ld.bfd: warning: arch/arm/boot/compressed/vmlinux has a LOAD segment with RWX permissions
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
```

Hình 3. Build Linux kernel zImage thành công

```
battousai1504@Dang-Laptop:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/boot/zImage
ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 battousai1504 battousai1504 4.9M Sep 20 15:27 arch/arm/boot/zImage
-rw-rw-r-- 1 battousai1504 battousai1504 14K Sep 20 15:22 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
```

Hình 4. Build Device Tree Blob cho Cortex-A9

3.4. Output của Part A

Artifact	Đường dẫn	Trạng thái
zImage	~/embedded_lab1/output/zImage	Đã build – 4.9 MB
Device Tree	~/embedded_lab1/output/vexpress-v2p-ca9.dtb	Đã build – 14 KB
Kernel config	~/embedded_lab1/configs/ kernel.config	Đã lưu – 96 KB

4. PHẦN B - U-BOOT VÀ KERNEL COMMAND LINE

4.1. Build U-Boot cho vexpress-a9

```
/usr/bin/arm-linux-gnueabihf-ld.bfd: warning: lib/efi_loader/dtbdump_efi.so has a LOAD segment with RWX permissions
  OBJCOPY lib/efi_loader/dtbdump.efi
  CC      lib/elf.o
  LD      lib/efi_loader/initrddump_efi.so
/usr/bin/arm-linux-gnueabihf-ld.bfd: warning: lib/efi_loader/initrddump_efi.so has a LOAD segment with RWX permissions
  OBJCOPY lib/efi_loader/initrddump.efi
  AR      fs/ubifs/built-in.o
  AR      fs/built-in.o
  AR      lib/built-in.o
  LD      u-boot
/usr/bin/arm-linux-gnueabihf-ld.bfd: warning: u-boot has a LOAD segment with RWX permissions
  OBJCOPY u-boot.srec
  OBJCOPY u-boot-nodtb.bin
  SYM     u-boot.sym
  COPY    u-boot.bin
```

```
cd ~/embedded_lab1/uboot/u-boot-2022.04
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make vexpress_ca9x4_defconfig
make -j"$(nproc)"
```

4.2. Boot vào U-Boot shell

```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -nographic \
  -kernel ~/embedded_lab1/output/u-boot
```

```
battousai1504@Dang-Laptop:~$ qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -nographic \
  -kernel "$LAB/output/u-boot" \
  2>&1 | tee "$LAB/logs/partB_uboot.log"


U-Boot 2022.04 (Sep 20 2026 - 16:03:29 +0700)

DRAM:  512 MiB
WARNING: Caches not enabled
Core:  12 devices, 7 uclasses, devicetree: embed
Flash: 64 MiB
MMC:   mmci@5000: 0
Loading Environment from Flash... *** Warning - bad CRC, using default environment

In:    serial
Out:   serial
Err:   serial
Net:   eth0: ethernet@3,02000000
Hit any key to stop autoboot:  0
=>
```

Minh chứng 4.2. U-Boot 2022.04 khởi động thành công, nhận 512 MiB DRAM và dừng tại dấu nhắc lệnh.

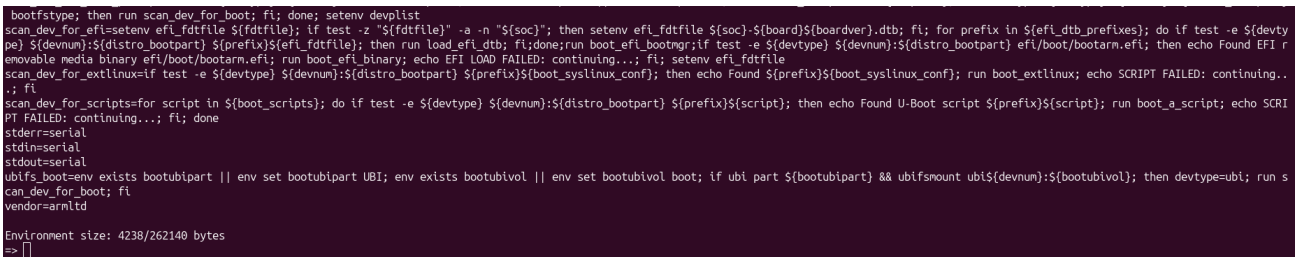
4.3. Thực hành environment variables

```
=> version
=> bdinfo
=> printenv
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init
mem=512M'
=> printenv bootargs
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
=> printenv bootcmd
```

```
=> version
U-Boot 2022.04 (Sep 20 2026 - 15:40:23 +0700)

arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
GNU ld (GNU Binutils for Ubuntu) 2.46
=> bdinfo
boot_params = 0x60002000
DRAM bank   = 0x00000000
-> start    = 0x60000000
-> size     = 0x20000000
DRAM bank   = 0x00000001
-> start    = 0x80000000
-> size     = 0x00000004
flashstart  = 0x40000000
flashsize   = 0x04000000
flashoffset = 0x00000000
baudrate    = 38400 bps
relocaddr   = 0x7ff6c000
reloc off   = 0x1f76c000
Build       = 32-bit
current eth = ethernet@3,02000000
ethaddr     = 52:54:00:12:34:56
IP addr     = <NULL>
fdt_blob    = 0x7ffe0430
new_fdt     = 0x00000000
fdt_size    = 0x00000000
lmb_dump_all:
 memory.cnt  = 0x1
 memory[0]      [0x60000000-0x80000003], 0x20000004 bytes flags: 0
 reserved.cnt  = 0x2
 reserved[0]    [0x4c000000-0x4c7fffff], 0x00800000 bytes flags: 4
 reserved[1]    [0x7fb27c88-0x7fffffff], 0x004d8378 bytes flags: 0
devicetree  = embed
arch_number = 0x000008e0
TLB addr    = 0x7fff0000
irq_sp      = 0x7fb2bec0
sp start    = 0x7fb2beb0
Early malloc usage: 370 / 400
```

```
=> printenv
arch=arm
baudrate=38400
board=vexpress
board_name=vexpress
boot_a_script=load ${devtype} ${devnum}:${distro_bootpart} ${scriptaddr} ${prefix}${script}; source ${scriptaddr}
boot_efi_binary=load ${devtype} ${devnum}:${distro_bootpart} ${kernel_addr_r} efi/boot/bootarm.efi; if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r};else bootefi ${kernel_addr_r} ${f
dtcontroladdr};fi
boot_efi_bootmgr=if fdt addr ${fdt_addr_r}; then bootefi bootmgr ${fdt_addr_r};else bootefi bootmgr;fi
boot_extlinux=sysboot ${devtype} ${devnum}:${distro_bootpart} any ${scriptaddr} ${prefix}${boot_syslinux_conf}
boot_prefixes=/ /boot/
boot_script_dhcp=boot.scr.uimg
boot_scripts=boot.scr.uimg boot.scr
boot_syslinux_conf=extlinux/extlinux.conf
boot_targets=mmc1 mmc0 pxe dhcp
bootargs=console=tty0 console=ttyAMA0,38400n8
bootcmd=run distro_bootcmd; run bootflash
bootcmd_dhcp=devtype=dhcp; if dhcp ${scriptaddr} ${boot_script_dhcp}; then source ${scriptaddr}; fi;setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; then setenv efi_fdtfile ${soc}-${
board}${boardver}.dtb; fi; setenv efi_old_vci ${bootp_vci};setenv efi_old_arch ${bootp_arch};setenv bootp_vci PXEClient:Arch:00010:UNDI:003000;setenv bootp_arch 0xa;if dhcp ${kernel_addr_r}; then tftpboot
 ${fdt_addr_r} dtb/${efi_fdtfile};if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_addr_r}; else bootefi ${kernel_addr_r} ${fdtcontroladdr};fi;fi;setenv bootp_vci ${efi_old_vci};setenv bootp
_arch ${efi_old_arch};setenv efi_fdtfile;setenv efi_old_arch;setenv efi_old_vci;
bootcmd_mmc0=devnum=0; run mmc_boot
bootcmd_mmc1=devnum=1; run mmc_boot
bootcmd_pxe=dhcp; if pxe get; then pxe boot; fi
bootdelay=2
bootflash=run flashargs; cp ${ramdisk_addr} ${ramdisk_addr_r} ${maxramdisk}; bootm ${kernel_addr} ${ramdisk_addr_r}
console=ttyAMA0,38400n8
cpu=armv7
distro_bootcmd=for target in ${boot_targets}; do run bootcmd_${target}; done
dram=1024M
efi_dtb_prefixes=/ /dtb/ /dtb/current/
ethaddr=52:54:00:12:34:56
fdt_addr_r=0x60000000
fdtcontroladdr=7ffe0430
fdtfile=vexpress-v2p-ca9.dtb
flashargs=setenv bootargs root=${root} console=${console} mem=${dram} mtdparts=${mtd} mmci.fmax=190000 devtmpfs.mount=0  vmalloc=256M
kernel_addr_r=0x60100000
load_efi_dtb=load ${devtype} ${devnum}:${distro_bootpart} ${fdt_addr_r} ${prefix}${efi_fdtfile}
loadaddr=0x90000000
mmc_boot=if mmc dev ${devnum}; then devtype=mmc; run scan_dev_for_boot_part; fi
mtd=armflash:1M@0x800000(uboot),7M@0x1000000(kernel),24M@0x2000000(initrd)
root=/dev/sda1 rw
scan_dev_for_boot=echo Scanning ${devtype} ${devnum}:${distro_bootpart}...; for prefix in ${boot_prefixes}; do run scan_dev_for_extlinux; run scan_dev_for_scripts; done;run scan_dev_for_efi;
scan_dev_for_boot_part=part list ${devtype} ${devnum} -bootable devplist; env exists devplist || setenv devplist 1; for distro_bootpart in ${devplist}; do if fstype ${devtype} ${devnum}:${distro_bootpart}
 bootfstype; then run scan_dev_for_boot; fi; done; setenv devplist
```



Đánh giá log thực tế: U-Boot đã khởi động thành công và xuất hiện dấu nhắc =>. Các dòng MMC Device 1 not found, TFTP error và Wrong Image Format xuất hiện vì không nhấn phím dừng autoboot nên bootcmd mặc định lần lượt thử MMC/DHCP/TFTP khi chưa có boot image trên các nguồn đó; đây không phải lỗi build U-Boot. Cảnh báo bad CRC chỉ cho biết QEMU flash chưa có environment hợp lệ nên U-Boot dùng default environment.



Minh chứng 4.3. Kết quả version, bdinfo, printenv và các biến địa chỉ/bootargs/bootcmd sau khi thiết lập.

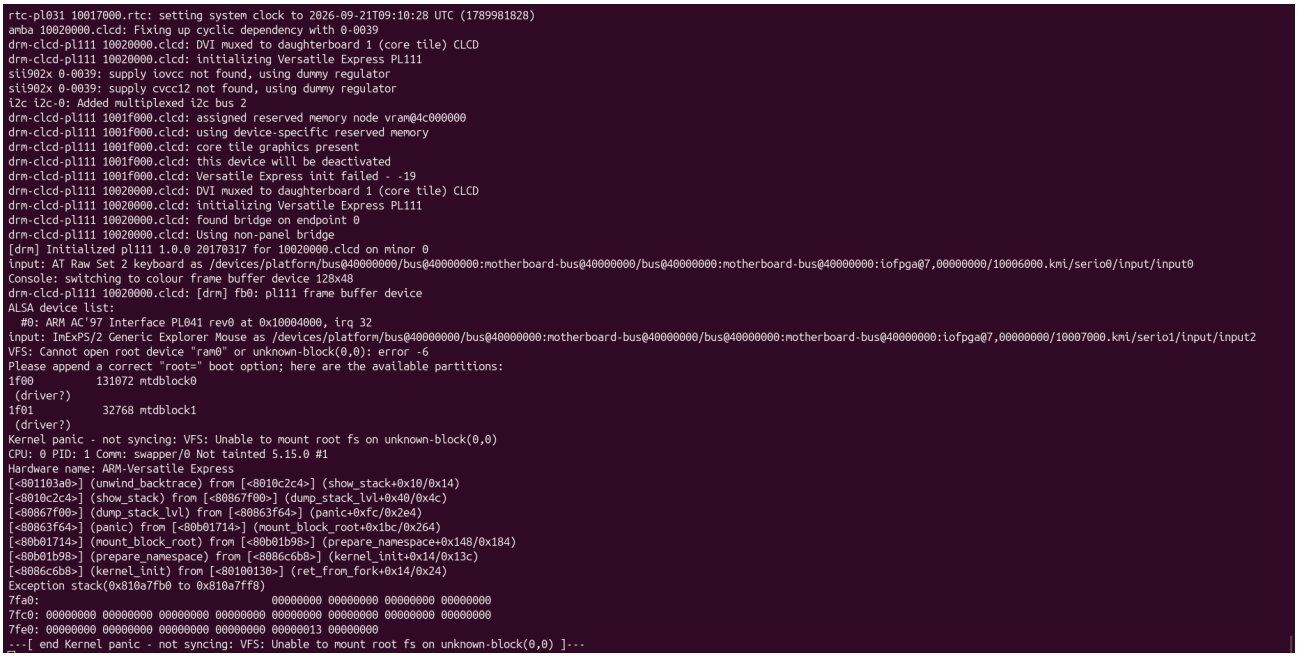
4.4. Giải thích bootargs

Tham số	Ý nghĩa
console=ttyAMA0,115200	Sử dụng PL011 UART0 làm kernel console, baud rate 115200.
root=/dev/ram0	Root filesystem đặt trên RAM disk trong phép thử trước khi có rootfs hoàn chỉnh.
rw	Mount root filesystem ở chế độ read/write.
init=/sbin/init	Chỉ định userspace PID 1 sau khi root filesystem sẵn sàng.
mem=512M	Giới hạn/khai báo RAM kernel sử dụng, khớp với QEMU -m 512M.

4.5. Thử boot kernel khi chưa có root filesystem

```
qemu-system-arm \
  -M vexpress-a9 -cpu cortex-a9 -m 512M -nographic \
  -kernel ~/embedded_lab1/output/zImage \
  -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -append 'console=ttyAMA0,115200 root=/dev/ram0 rw mem=512M'
```

Mục đích phép thử là chứng minh kernel và DTB đã chạy đến giai đoạn mount root filesystem. Khi chưa cung cấp initramfs/rootfs hợp lệ, log thực tế dừng tại: “Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)”. Đây là failure có chủ đích để xác định boundary giữa kernel-space và userspace.



Hình 5. Kernel panic khi chưa có root filesystem

5. PHẦN C - BUSYBOX, ROOT FILESYSTEM VÀ INITRAMFS

5.1. Build BusyBox static

```
cd ~/embedded_lab1/rootfs/busybox-1.35.0
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make defconfig
make HOSTCC='gcc -std=gnu89' menuconfig   # bật CONFIG_STATIC, tắt CONFIG_TC
make -j"$(nproc)"
make install
file _install/bin/busybox
```

Static linking giúp binary BusyBox tự chứa phần lớn code thư viện cần thiết, giảm phụ thuộc vào dynamic loader và shared libraries trong một initramfs tối giản.

```
ĐÃ XÁC MINH TRÊN HOST
file ~/embedded_lab1/rootfs/busybox-1.35.0/_install/bin/busybox
ELF 32-bit LSB executable, ARM, EABI5, statically linked, stripped
busybox.config: CONFIG_STATIC=y; CROSS_COMPILER_PREFIX=arm-linux-gnueabihf-; CONFIG_TC
is not set
```

5.2. Cấu trúc root filesystem

```
initramfs/
├── bin/
├── sbin/
├── usr/
├── dev/
├── proc/
├── sys/
├── tmp/
├── root/
├── etc/
│   ├── inittab
```

```
│   └── init.d/rcS
├── init
└── linuxrc -> bin/busybox
```

```
$ ls -F ~/embedded_lab1/rootfs/initramfs
bin/  dev/  etc/  init*  linuxrc@  proc/  root/  sbin/  sys/  tmp/  usr/
```

Các thư mục được tạo theo cấu trúc tối giản của Linux userspace. /proc và /sys là mount point cho pseudo-filesystem của kernel; /dev chứa device nodes cần thiết; /etc chứa cấu hình init.

5.3. File /init, inittab và rcS

```
# /init
#!/bin/sh
exec /sbin/init

# /etc/inittab (các dòng chính)
::sysinit:/etc/init.d/rcS
::askfirst:-/bin/sh

# /etc/init.d/rcS
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t tmpfs none /tmp
hostname embedded-lab
echo "Boot completed! Welcome to Embedded Linux Lab 1"
```

5.4. Đóng gói initramfs

```
cd ~/embedded_lab1/rootfs/initramfs
find . | cpio -H newc -o --owner=0:0 | gzip -9 >
~/embedded_lab1/output/initramfs.cpio.gz

zcat ~/embedded_lab1/output/initramfs.cpio.gz | cpio -t | head -30
```

```
battousai1504@Dang-Laptop:~/embedded_lab1/rootfs/initramfs$ gzip -t "$LAB/output/initramfs.cpio.gz"

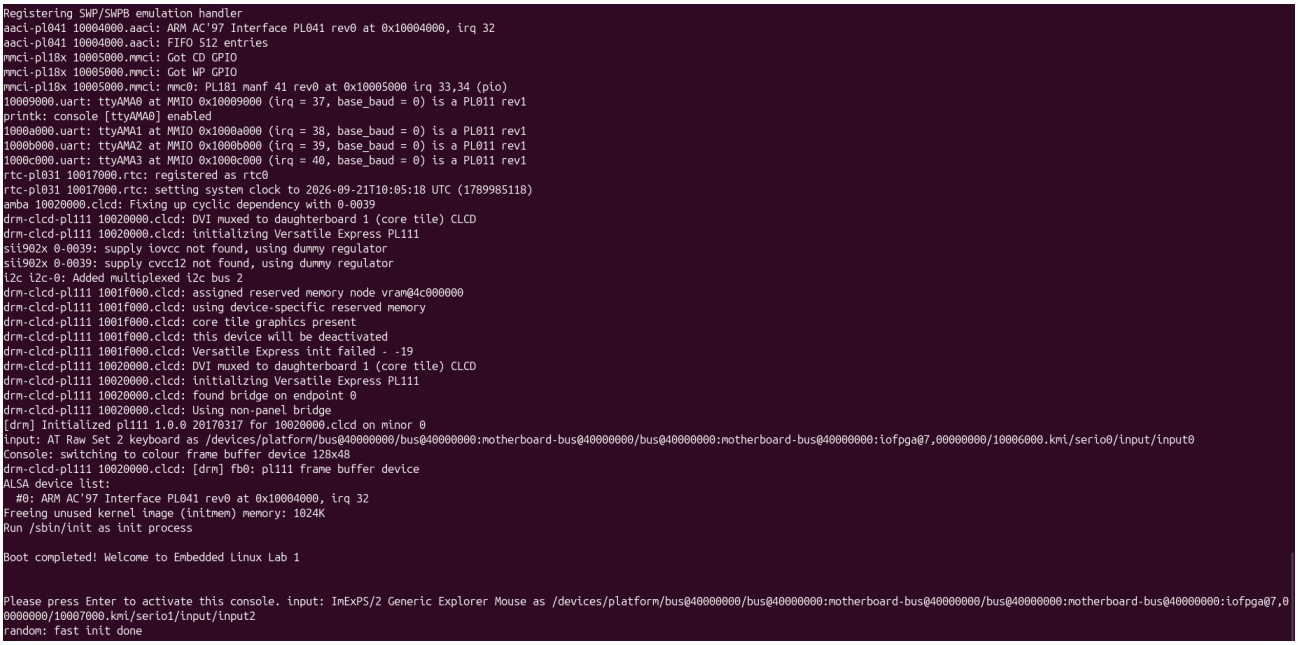
gzip -dc "$LAB/output/initramfs.cpio.gz" \
  | cpio -t \
  | head -30

ls -lh "$LAB/output/initramfs.cpio.gz"
.
sys
root
bin
bin/sed
bin/mountpoint
bin/reformime
bin/touch
bin/chattr
bin/getopt
bin/umount
bin/base64
bin/netstat
bin/hush
bin/setarch
bin/fatattr
bin/lzop
bin/grep
bin/echo
bin/chown
bin/watch
bin/uname
bin/printenv
bin/cp
bin/date
bin/ipcalc
bin/run-parts
bin/fgrep
bin/fsync
bin/iostat
3201 blocks
-rw-rw-r-- 1 battousai1504 battousai1504 1.1M Sep 21 17:02 /home/battousai1504/embedded_lab1/output/initramfs.cpio.gz
```

Minh chứng 5.4. Archive initramfs.cpio.gz hợp lệ, chứa cây thư mục BusyBox và có kích thước khoảng 1,1 MiB.

5.5. Boot hệ thống hoàn chỉnh

```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -smp 2 \
  -nographic \
  -kernel ~/embedded_lab1/output/zImage \
  -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -initrd ~/embedded_lab1/output/initramfs.cpio.gz \
  -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
```



Kết quả boot xác nhận kernel đã giải nén initramfs, chạy /sbin/init, thực thi rcS và hiển thị thông báo “Boot completed! Welcome to Embedded Linux Lab 1”. Dấu nhắc console xuất hiện, chứng minh chuỗi kernel → PID 1 → BusyBox userspace hoạt động hoàn chỉnh.

Minh chứng 5.5. Kernel 5.15 boot thành công với initramfs và chuyển quyền điều khiển sang BusyBox init.

5.6. Xác minh hệ thống sau boot

```
uname -a
cat /proc/cpuinfo
cat /proc/meminfo | head -5
mount
ps
cat /sys/devices/system/cpu/possible
cat /sys/devices/system/cpu/online
```

```
/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Sun Sep 20 14:58:20 +07 2026 armv7l GNU/Linux
/ # cat /proc/cpuinfo
processor       : 0
model name      : ARMv7 Processor rev 0 (v7l)
BogoMIPS        : 1699.02
Features        : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer : 0x41
CPU architecture: 7
CPU variant     : 0x0
CPU part        : 0xc09
CPU revision    : 0

processor       : 1
model name      : ARMv7 Processor rev 0 (v7l)
BogoMIPS        : 1699.02
Features        : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer : 0x41
CPU architecture: 7
CPU variant     : 0x0
CPU part        : 0xc09
CPU revision    : 0

Hardware        : ARM-Versatile Express
Revision        : 0000
Serial          : 0000000000000000
/ # head -5 /proc/meminfo
MemTotal:         508288 kB
MemFree:          492912 kB
MemAvailable:     490148 kB
Buffers:               0 kB
Cached:            1572 kB
/ # mount
rootfs on / type rootfs (rw,size=244912k,nr_inodes=61228)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
tmpfs on /tmp type tmpfs (rw,relatime)
/ # ps
PID   USER     TIME  COMMAND
    1 root      0:02 init
    2 root      0:00 [kthreadd]
    3 root      0:00 [rcu_gp]
    4 root      0:00 [rcu_par_gp]
    5 root      0:00 [kworker/0:0-eve]
    6 root      0:00 [kworker/0:0H]
```

```
    7 root      0:00 [kworker/u8:0-ev]
    8 root      0:00 [mm_percpu_wq]
    9 root      0:00 [ksoftirqd/0]
   10 root      0:00 [rcu_sched]
   11 root      0:00 [migration/0]
   12 root      0:00 [cpuhp/0]
   13 root      0:00 [cpuhp/1]
   14 root      0:00 [migration/1]
   15 root      0:00 [ksoftirqd/1]
   16 root      0:00 [kworker/1:0-rcu]
   17 root      0:00 [kworker/1:0H-ev]
   18 root      0:00 [cpuhp/2]
   19 root      0:00 [migration/2]
   20 root      0:00 [ksoftirqd/2]
   21 root      0:00 [kworker/2:0]
   22 root      0:00 [kworker/2:0H]
   23 root      0:00 [cpuhp/3]
   24 root      0:00 [kworker/0:1]
   25 root      0:00 [migration/3]
   26 root      0:00 [ksoftirqd/3]
   27 root      0:00 [kworker/3:0]
   28 root      0:00 [kworker/3:0H]
   29 root      0:00 [kdevtmpfs]
   30 root      0:00 [inet_frag_wq]
   31 root      0:00 [kworker/1:1-eve]
   32 root      0:00 [khungtaskd]
   33 root      0:00 [oom_reaper]
   34 root      0:00 [writeback]
   35 root      0:00 [kcompactd0]
   36 root      0:00 [kblockd]
   37 root      0:00 [ata_sff]
   38 root      0:00 [kworker/u8:1-ev]
   39 root      0:00 [rpciod]
   40 root      0:00 [kworker/u9:0]
   41 root      0:00 [xprtiod]
   42 root      0:00 [kworker/1:1H]
   43 root      0:00 [kswapd0]
   44 root      0:00 [nfsiod]
   45 root      0:00 [kworker/1:2-eve]
   46 root      0:00 [kworker/u8:2-ev]
   50 root      0:00 [irq/33-mmci-pl1]
   51 root      0:00 [kworker/1:3-rcu]
   52 root      0:00 [card0-crtc0]
   58 root      0:00 -/bin/sh
   63 root      0:00 ps
```

```
/ # cat /sys/devices/system/cpu/possible
0-3
/ # cat /sys/devices/system/cpu/online
0-1
/ #
```

Kết quả sau boot: uname xác nhận armv7l; /proc/cpuinfo nhận hai CPU Cortex-A9 (implementer 0x41, part 0xc09); MemTotal xấp xỉ 512 MiB; rootfs, proc, sysfs và tmpfs đã mount; tiến trình init và shell đang chạy. Giá trị possible=0-3 phản ánh topology tối đa trong DTB, còn online=0-1 khớp với QEMU -smp 2.

6. LOG THỰC HÀNH VÀ BẰNG CHỨNG

Bảng dưới đây đối chiếu trực tiếp từng bằng chứng bắt buộc của đề Lab 01 với vị trí minh chứng trong báo cáo.

Phần	Bằng chứng bắt buộc	Trạng thái	Vị trí trong báo cáo
A	Output cấu hình VExpress / defconfig	**ĐÃ CÓ**	Hình 2
A	Log cuối build kernel và ls -lh zImage	**ĐÃ CÓ**	Hình 3–4
A	Build vexpress-v2p-ca9.dtb	**ĐÃ CÓ**	Hình 4
B	printenv / setenv bootargs / bootcmd	**ĐÃ CÓ**	Mục 4.3
B	Kernel panic khi chưa có rootfs	**ĐÃ CÓ**	Hình 5
C	BusyBox static và nội dung initramfs	**ĐÃ CÓ**	Mục 5.1, 5.4
C	QEMU boot thành công tới BusyBox shell	**ĐÃ CÓ**	Mục 5.5
C	uname -a / cpuinfo / meminfo / mount / ps	**ĐÃ CÓ**	Mục 5.6
C	So sánh boot log -smp 1 và -smp 2	**ĐÃ CÓ**	Câu 5

7. PHÂN TÍCH VÀ TRẢ LỜI CÂU HỎI

Câu 1. So sánh arm-linux-gnueabi và arm-linux-gnueabihf

Hai toolchain khác nhau chủ yếu ở ABI dùng cho floating-point trong userspace. arm-linux-gnueabi dùng soft-float calling convention: tham số/kết quả floating-point được truyền theo quy ước general-purpose registers/stack. arm-linux-gnueabihf dùng hard-float ABI, cho phép truyền floating-point qua VFP registers theo AAPCS-VFP khi target và thư viện hỗ trợ.

Cortex-A9 thuộc ARMv7-A và hỗ trợ VFP, vì vậy bài lab sử dụng gnueabihf để đồng bộ với một userspace ARM hard-float. Tuy nhiên cần phân biệt: Linux kernel không phụ thuộc vào hard-float userspace ABI theo cùng cách với ứng dụng. Kernel thông thường tránh sử dụng floating-point registers trong đường thực thi thường. Vì thế compiler prefix gnueabi vẫn có thể build một kernel ARMv7 nếu ISA flags/config đúng; vấn đề lớn hơn thường nằm ở tương thích userspace và shared libraries khi trộn soft-float với hard-float.

Ý nghĩa thực tế: toolchain, sysroot và userspace binary phải thống nhất ABI. Nếu BusyBox được build dynamic theo hard-float nhưng rootfs chỉ có soft-float loader/libraries (hoặc ngược lại), binary có thể không chạy dù file tồn tại.

Câu 2. Memory map của vexpress-a9 và địa chỉ load

VExpress-A9 bố trí DRAM tại vùng bắt đầu khoảng 0x60000000, trong khi vùng địa chỉ thấp được dành cho flash/alias và các peripheral/platform region. Vì vậy image được nạp vào các địa chỉ thuộc vùng RAM hợp lệ, ví dụ DTB 0x60000000, kernel 0x60100000 và initramfs 0x64000000 theo bài lab.

Các địa chỉ phải vừa nằm trong DRAM, vừa không overlap nhau. Nếu bootcmd trỏ sai địa chỉ, U-Boot/kernel có thể đọc nhầm dữ liệu, ghi vào MMIO/unmapped region, làm corrupt image hoặc gặp Data Abort/hang. Do đó memory map là ràng buộc vật lý của hệ thống chứ không chỉ là tham số phần mềm tùy ý.

Câu 3. Phân tích MIDR 0x410fc090

Trường	Giá trị	Ý nghĩa
Implementer	0x41	ARM Limited
Variant	0x0	Variant 0
Architecture	0xF	Architecture field dùng CPUID scheme
Part Number	0xC09	ARM Cortex-A9
Revision	0x0	Revision 0

MIDR giúp kernel nhận diện loại CPU để chọn behavior/errata phù hợp. Part number 0xC09 là dấu hiệu quan trọng xác nhận CPU model Cortex-A9 trong môi trường QEMU của bài lab.

Câu 4. Tại sao BusyBox nên static link trong initramfs tối giản?

Static BusyBox giảm số thành phần bắt buộc trong rootfs: kernel chỉ cần load một ELF ARM executable mà không phải tìm dynamic loader và chuỗi shared libraries. Điều này đặc biệt phù hợp với initramfs nhỏ, dễ debug và ít lỗi dependency.

Nếu build BusyBox dynamic cho ARM hard-float, cần copy dynamic loader và các thư viện được binary khai báo trong ELF dynamic section. Các file thường gặp trong Ubuntu ARM cross sysroot gồm:

- /usr/arm-linux-gnueabihf/lib/ld-linux-armhf.so.3 - ELF dynamic loader.
- /usr/arm-linux-gnueabihf/lib/libc.so.6 - GNU C Library.
- /usr/arm-linux-gnueabihf/lib/libm.so.6 - math library (nếu binary/config yêu cầu).

Danh sách chính xác nên được xác nhận bằng arm-linux-gnueabihf-readelf -d <binary> | grep NEEDED và readelf -l <binary> | grep interpreter, vì dependency có thể thay đổi theo cấu hình BusyBox/toolchain.

Câu 5. Ảnh hưởng của SMP khi chạy -smp 2

Khi bật SMP và QEMU tạo 2 Cortex-A9 CPU, kernel không chỉ khởi tạo CPU0 mà còn phải bring-up secondary CPU, tạo per-CPU data, idle thread, scheduler state, timer/interrupt context và cơ chế inter-processor interrupt/synchronization.

Log thực tế cho thấy với -smp 1, kernel báo “Brought up 1 node, 1 CPU” và “Total of 1 processors activated”. Với -smp 2, log xuất hiện thêm “CPU1: thread -1, cpu 1, socket 0, mpidr 80000001”, sau đó đổi thành “Brought up 1 node, 2 CPUs” và “Total of 2 processors activated”. Các dòng này chứng minh secondary core CPU1 đã được bring-up và chuyển sang trạng thái online.

```
battousai1504@Dang-Laptop:~/embedded_lab1/rootfs/initramfs$ grep -E 'Bringing up|CPU1:|Brought up|Total of' \
  "$LAB/logs/partC_smp1.log" \
  "$LAB/logs/partC_smp2.log"
/home/battousai1504/embedded_lab1/logs/partC_smp1.log:smp: Bringing up secondary CPUs ...
/home/battousai1504/embedded_lab1/logs/partC_smp1.log:smp: Brought up 1 node, 1 CPU
/home/battousai1504/embedded_lab1/logs/partC_smp1.log:SMP: Total of 1 processors activated (1567.94 BogoMIPS).
/home/battousai1504/embedded_lab1/logs/partC_smp2.log:smp: Bringing up secondary CPUs ...
/home/battousai1504/embedded_lab1/logs/partC_smp2.log:CPU1: thread -1, cpu 1, socket 0, mpidr 80000001
/home/battousai1504/embedded_lab1/logs/partC_smp2.log:CPU1: Spectre v2: using BPIALL workaround
/home/battousai1504/embedded_lab1/logs/partC_smp2.log:smp: Brought up 1 node, 2 CPUs
/home/battousai1504/embedded_lab1/logs/partC_smp2.log:SMP: Total of 2 processors activated (3398.04 BogoMIPS).
```

8. NHẬN XÉT, KHÓ KHĂN VÀ DEBUG

8.1. Lỗi 1 - Assembler báo không hỗ trợ instruction ARMv7

Triệu chứng thực tế:

```
Error: selected processor does not support `dmb ish' in ARM mode
Error: selected processor does not support `dsb st' in ARM mode
Error: selected processor does not support `ldrex ...' in ARM mode
Error: architectural extension `mp' is not allowed for the current base architecture
```

Phân tích nguyên nhân: kernel 5.15 cần ARMv7/SMP instructions, nhưng compile command thực tế chứa đồng thời -D__LINUX_ARM_ARCH__=7, -march=armv5t và -Wa,-march=armv7-a. Với GCC 15 hard-float, phép thử cc-option cho -march=armv7-a có thể thất bại vì thiếu FP extension; Kbuild rơi về ARMv5T, trong khi source vẫn sinh DMB/DSB/LDREX/STREX/SEV/CPSID.

Cách xử lý đã thực hiện: giữ nguyên source kernel, xóa các CFLAGS/KCFLAGS môi trường không phù hợp và truyền CPU ISA tương thích GCC 15 ở mức KCFLAGS. Kernel vẫn giữ -msoft-float của Kbuild; “+fp” ở đây giúp compiler chấp nhận target Cortex-A9, không biến kernel thành userspace hard-float ABI.

```
make -j2 KCFLAGS=-march=armv7-a+fp zImage dtbs
```

Xác minh: Build không còn dừng ở các instruction ARMv7; make zImage thành công và make dtbs tạo vexpress-v2p-ca9.dtb thành công.

8.2. Các lỗi thực tế khác và cách xử lý

Bằng chứng debug được ghi từ đúng log và cấu hình đã gặp trong quá trình thực hành.

Lỗi 2 – make menuconfig báo thiếu ncurses dù libncurses-dev đã cài

Triệu chứng: *** Unable to find the ncurses libraries or the required header files.

Nguyên nhân: BusyBox 1.35 dùng phép thử lxdialog cũ với hàm main() không khai báo kiểu trả về. GCC 15 xem implicit-int là lỗi, làm bước probe thất bại và tạo chẩn đoán nhầm là thiếu ncurses.

Cách xử lý: Chạy make HOSTCC="gcc -std=gnu89" menuconfig (hoặc sửa probe thành int main(void) { return 0; }).

Xác minh: menuconfig mở được; cấu hình BusyBox được lưu vào busybox.config.

Lỗi 3 – BusyBox applet tc không build với Linux UAPI headers hiện tại

Triệu chứng: networking/tc.c: error: ‘TCA_CBQ_MAX’ undeclared (kèm các lỗi CBQ tương tự).

Nguyên nhân: BusyBox 1.35 tc applet dùng CBQ definitions cũ, không còn tương thích với bộ Linux userspace headers hiện tại. Đây là lỗi tương thích source/header, không phải lỗi ARM toolchain.

Cách xử lý: Tắt CONFIG_TC trong BusyBox menuconfig/.config rồi build lại. Applet này không cần cho initramfs tối giản của bài lab.

Xác minh: _install/bin/busybox được tạo thành công; file xác nhận ELF 32-bit ARM EABI5, statically linked.

Lỗi 4 – U-Boot saveenv báo địa chỉ không nằm trên sector boundary

Triệu chứng: Saving Environment to Flash... Error: start and/or end address not on sector boundary; Failed (1).

Nguyên nhân: Cấu hình hiện tại đặt CONFIG_ENV_ADDR=0x47F80000, trong khi NOR Bank 2 của QEMU chỉ ở 0x44000000–0x45FFFFFF; vùng environment nằm ngoài flash hợp lệ. ENV_SIZE và ENV_SECT_SIZE đều là 0x40000.

Cách xử lý: Đối với bài lab, dùng setenv trong phiên hiện tại và chấp nhận biến mất khi thoát QEMU. Nếu bắt buộc lưu bền vững, đặt ENV_ADDR vào sector hợp lệ, ví dụ 0x45FC0000, rồi rebuild U-Boot.

Xác minh: printenv hiển thị bootargs/bootcmd đúng trong cùng phiên; U-Boot vẫn chạy bình thường tới dấu nhắc =>.

8.3. Phương pháp debug rút ra

- Đọc lỗi đầu tiên và compile command thay vì chỉ đọc hàng trăm lỗi cascade ở cuối log.
- Khoanh vùng theo tầng: toolchain/config -> build artifact -> bootloader -> kernel -> rootfs -> PID 1/userspace.
- Dùng log và lệnh kiểm chứng (file, readelf, grep .config, /proc, /sys) thay vì suy đoán.

- Mỗi lần sửa chỉ thay đổi một nhóm biến để xác định quan hệ nhân-quả và tránh tạo lỗi mới.

9. KẾT LUẬN

Lab 01 đã hoàn thành đầy đủ ba phần. Part A tạo thành công zImage và vexpress-v2p-ca9.dtb cho ARMv7-A; Part B build và vận hành U-Boot, thiết lập bootargs/bootcmd, đồng thời kiểm chứng boundary kernel–rootfs bằng phép thử kernel panic có chủ đích; Part C build BusyBox static, tạo initramfs và boot thành công tới BusyBox shell trên QEMU vexpress-a9. Các lệnh uname, cpuinfo, meminfo, mount, ps và trạng thái CPU xác nhận kernel, filesystem và userspace hoạt động đúng với cấu hình Cortex-A9, 512 MiB RAM và SMP hai CPU.

Quá trình xử lý lỗi cho thấy cần debug theo từng tầng: toolchain/Kbuild → artifact → bootloader → kernel/DTB → rootfs → PID 1. Việc đọc lỗi đầu tiên, kiểm tra compile flags và xác minh bằng log thực tế giúp phân biệt lỗi nguồn, lỗi ABI, lỗi cấu hình và lỗi môi trường; đây là phương pháp có thể áp dụng trực tiếp cho các dự án Embedded Linux thực tế.

10. CẤU TRÚC FILE NỘP BÀI

```
MSSV_HoTen_Lab01/
├── output/
│   ├── zImage
│   ├── vexpress-v2p-ca9.dtb
│   └── initramfs.cpio.gz
├── configs/
│   ├── kernel.config
│   └── busybox.config
├── rootfs/
│   └── initramfs/
└── bao_cao/
    └── MSSV_Lab01_BaoCao.pdf
```

11. TÀI LIỆU THAM KHẢO

[1] Tài liệu Lab 01 - Embedded Linux Systems (LOS201): Cấu hình Kernel & Khởi động Hệ thống Linux Nhúng (ARMv7).

[2] LOS201 Lecture Slides - Chapter 4: Linux Kernel.

[3] LOS201 Lecture Slides - Chapter 5: Kernel Initialization.

[4] LOS201 Lecture Slides - Chapter 6: User Space Initialization.

[5] LOS201 Lecture Slides - Chapter 7: Bootloaders.

[6] Linux Kernel 5.15 source tree and kernel documentation.

[7] U-Boot 2022.04 documentation.

[8] BusyBox 1.35.0 documentation.

[9] QEMU ARM Versatile Express documentation.